VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 6: MongoDB and Basics commands

Branch/ Class: B.Tech/M.Tech
Faculty Name: Prof. S.Gopikrishnan
Student name: VOONNA TARUN KUMAR

Date:11-03-2026
School: SCOPE
Reg. no.: 23BCE9867

1. Use MongoDB to implement the following DB operations

1. Create a database called 'vehicles' and *write* a MongoDB query to select database as "vehicles".

```
> use vehicles
< switched to db vehicles
```

2. Write a MongoDB query to display all the databases.

```
> show dbs
< admin      40.00 KiB
  config    108.00 KiB
  local      88.00 KiB
  test       72.00 KiB
  vehicles   80.00 KiB
```

3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

```
> db.createCollection("two_wheelers", { capped: true, size: 1048576 })
  db.createCollection("four_wheelers")
< { ok: 1 }
```

4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array,

manufacturer, performance (out of 10), timestamp (date and year release) and price

```
> db.two_wheelers.insertMany([
    {
      bike_name: "Pulsar NS200",
      model: "gear",
      category: "200cc",
      colors_available: ["red", "black", "blue"],
      manufacturer: "Bajaj",
      performance: 8.5,
      timestamp: ISODate("2023-01-15"),
      price: 150000
    },
    {
      bike_name: "Shine",
      model: "gearless",
      category: "125cc",
      colors_available: ["red", "black"],
      manufacturer: "Honda",
      performance: 7.2,
      timestamp: ISODate("2022-06-10"),
      price: 85000
    },
```

```
    {
      bike_name: "Apache RTR 160",
      model: "gear",
      category: "160cc",
      colors_available: ["sport red", "black", "blue"],
      manufacturer: "TVS",
      performance: 8.0,
      timestamp: ISODate("2023-03-20"),
      price: 125000
    },
    {
      bike_name: "Activa 6G",
      model: "gearless",
      category: "110cc",
      colors_available: ["red", "white", "black"],
      manufacturer: "Honda",
      performance: 7.8,
      timestamp: ISODate("2021-11-01"),
      price: 78000
    },
    {
      bike_name: "FZS FI V4",
      model: "gear",
      category: "150cc",
      colors_available: ["dark blue", "red"],
      manufacturer: "Yamaha",
      performance: 8.2,
      timestamp: ISODate("2023-09-15"),
      price: 135000
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b133d04fee05ee9c0e48ab'),
    '1': ObjectId('69b133d04fee05ee9c0e48ac'),
    '2': ObjectId('69b133d04fee05ee9c0e48ad'),
    '3': ObjectId('69b133d04fee05ee9c0e48ae'),
    '4': ObjectId('69b133d04fee05ee9c0e48af')
  }
}
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
> db.four_wheelers.insertMany([
  {
    vehicle_name: "Swift",
    model: "own",
    category: "car",
    variants: ["VXi", "ZXi", "petrol"],
    manufacturer: "Maruti Suzuki",
    performance: 7.5,
    timestamp: ISODate("2023-02-10"),
    price: 750000
  },
  {
    vehicle_name: "Eicher Pro 2049",
    model: "commercial",
    category: "mini truck",
    variants: ["diesel", "CNG"],
    manufacturer: "Eicher",
    performance: 6.8,
    timestamp: ISODate("2022-08-15"),
    price: 1250000
  },
  {
    vehicle_name: "Creta",
    model: "own",
    category: "car",
    variants: ["petrol", "diesel", "SX"],
    manufacturer: "Hyundai",
    performance: 8.3,
    timestamp: ISODate("2023-01-05"),
```

```
    vehicle_name: "Ashok Leyland Viking",
    model: "commercial",
    category: "bus",
    variants: ["diesel", "12m"],
    manufacturer: "Ashok Leyland",
    performance: 7.0,
    timestamp: ISODate("2022-12-20"),
    price: 2800000
  },
  {
    vehicle_name: "Thar",
    model: "own",
    category: "car",
    variants: ["diesel", "petrol", "LX"],
    manufacturer: "Mahindra",
    performance: 8.7,
    timestamp: ISODate("2023-10-10"),
    price: 1400000
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b133d84fee05ee9c0e48b0'),
    '1': ObjectId('69b133d84fee05ee9c0e48b1'),
    '2': ObjectId('69b133d84fee05ee9c0e48b2'),
    '3': ObjectId('69b133d84fee05ee9c0e48b3'),
    '4': ObjectId('69b133d84fee05ee9c0e48b4')
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
> db.two_wheelers.find().pretty()
< {
    _id: ObjectId('69b133d04fee05ee9c0e48ab'),
    bike_name: 'Pulsar NS200',
    model: 'gear',
    category: '200cc',
    colors_available: [
      'red',
      'black',
      'blue'
    ],
    manufacturer: 'Bajaj',
    performance: 8.5,
    timestamp: 2023-01-15T00:00:00.000Z,
    price: 150000
  }
  {
    _id: ObjectId('69b133d04fee05ee9c0e48ac'),
    bike_name: 'Shine',
    model: 'gearless',
    category: '125cc',
    colors_available: [
      'red',
      'black'
    ],
    manufacturer: 'Honda',
    performance: 7.2,
    timestamp: 2022-06-10T00:00:00.000Z,
    price: 85000
```

```
    _id: ObjectId('69b133d04fee05ee9c0e48ad'),
    bike_name: 'Apache RTR 160',
    model: 'gear',
    category: '160cc',
    colors_available: [
      'sport red',
      'black',
      'blue'
    ],
    manufacturer: 'TVS',
    performance: 8,
    timestamp: 2023-03-20T00:00:00.000Z,
    price: 125000
  }
  {
    _id: ObjectId('69b133d04fee05ee9c0e48ae'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '110cc',
    colors_available: [
      'red',
      'white',
      'black'
    ],
    manufacturer: 'Honda',
    performance: 7.8,
    timestamp: 2021-11-01T00:00:00.000Z,
    price: 78000
  }
```

```
> db.four_wheelers.find().pretty()
< {
    _id: ObjectId('69b133d84fee05ee9c0e48b0'),
    vehicle_name: 'Swift',
    model: 'own',
    category: 'car',
    variants: [
      'VXi',
      'ZXi',
      'petrol'
    ],
    manufacturer: 'Maruti Suzuki',
    performance: 7.5,
    timestamp: 2023-02-10T00:00:00.000Z,
    price: 750000
  }
  {
    _id: ObjectId('69b133d84fee05ee9c0e48b1'),
    vehicle_name: 'Eicher Pro 2049',
    model: 'commercial',
    category: 'mini truck',
    variants: [
      'diesel',
      'CNG'
    ],
```

```
  {
    _id: ObjectId('69b133d84fee05ee9c0e48b3'),
    vehicle_name: 'Ashok Leyland Viking',
    model: 'commercial',
    category: 'bus',
    variants: [
      'diesel',
      '12m'
    ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: 2022-12-20T00:00:00.000Z,
    price: 2800000
  }
  {
    _id: ObjectId('69b133d84fee05ee9c0e48b4'),
    vehicle_name: 'Thar',
    model: 'own',
    category: 'car',
    variants: [
      'diesel',
      'petrol',
      'LX'
    ],
    manufacturer: 'Mahindra',
    performance: 8.7,
    timestamp: 2023-10-10T00:00:00.000Z,
    price: 1400000
  }
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
> db.two_wheelers.find({}, { bike_name: 1, price: 1, _id: 0 })
< {
    bike_name: 'Pulsar NS200',
    price: 150000
  }
  {
    bike_name: 'Shine',
    price: 85000
  }
  {
    bike_name: 'Apache RTR 160',
    price: 125000
  }
  {
    bike_name: 'Activa 6G',
    price: 78000
  }
  {
    bike_name: 'FZS FI V4',
    price: 135000
  }
```

```
> db.four_wheelers.find({}, { vehicle_name: 1, price: 1, _id: 0 })
< {
    vehicle_name: 'Swift',
    price: 750000
  }
  {
    vehicle_name: 'Eicher Pro 2049',
    price: 1250000
  }
  {
    vehicle_name: 'Creta',
    price: 1150000
  }
  {
    vehicle_name: 'Ashok Leyland Viking',
    price: 2800000
  }
  {
    vehicle_name: 'Thar',
    price: 1400000
  }
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
> db.two_wheelers.find({ manufacturer: "Honda" }).pretty()
< {
    _id: ObjectId('69b133d04fee05ee9c0e48ac'),
    bike_name: 'Shine',
    model: 'gearless',
    category: '125cc',
    colors_available: [
      'red',
      'black'
    ],
    manufacturer: 'Honda',
    performance: 7.2,
    timestamp: 2022-06-10T00:00:00.000Z,
    price: 85000
  }
  {
    _id: ObjectId('69b133d04fee05ee9c0e48ae'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '110cc',
    colors_available: [
      'red',
      'white',
      'black'
    ],
    manufacturer: 'Honda',
    performance: 7.8,
    timestamp: 2021-11-01T00:00:00.000Z,
    price: 78000
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
> db.four_wheelers.find({ variants: "diesel" }).pretty()
< {
    _id: ObjectId('69b133d84fee05ee9c0e48b1'),
    vehicle_name: 'Eicher Pro 2049',
    model: 'commercial',
    category: 'mini truck',
    variants: [
      'diesel',
      'CNG'
    ],
    manufacturer: 'Eicher',
    performance: 6.8,
    timestamp: 2022-08-15T00:00:00.000Z,
    price: 1250000
  }
  {
    _id: ObjectId('69b133d84fee05ee9c0e48b2'),
    vehicle_name: 'Creta',
    model: 'own',
    category: 'car',
    variants: [
      'petrol',
      'diesel',
      'SX'
    ],
    manufacturer: 'Hyundai',
    performance: 8.3,
    timestamp: 2023-01-05T00:00:00.000Z,
```

```
{
  _id: ObjectId('69b133d84fee05ee9c0e48b3'),
  vehicle_name: 'Ashok Leyland Viking',
  model: 'commercial',
  category: 'bus',
  variants: [
    'diesel',
    '12m'
  ],
  manufacturer: 'Ashok Leyland',
  performance: 7,
  timestamp: 2022-12-20T00:00:00.000Z,
  price: 2800000
}
{
  _id: ObjectId('69b133d84fee05ee9c0e48b4'),
  vehicle_name: 'Thar',
  model: 'own',
  category: 'car',
  variants: [
    'diesel',
    'petrol',
    'LX'
  ],
  manufacturer: 'Mahindra',
  performance: 8.7,
  timestamp: 2023-10-10T00:00:00.000Z,
  price: 1400000
}
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
> db.two_wheelers.find(
    { performance: { $gt: 5 } },
    { bike_name: 1, category: 1, manufacturer: 1, _id: 0 }
  ).pretty()

  db.four_wheelers.find(
    { performance: { $gt: 5 } },
    { vehicle_name: 1, category: 1, manufacturer: 1, _id: 0 }
  ).pretty()
< {
    vehicle_name: 'Swift',
    category: 'car',
    manufacturer: 'Maruti Suzuki'
  }
  {
    vehicle_name: 'Eicher Pro 2049',
    category: 'mini truck',
    manufacturer: 'Eicher'
  }
  {
    vehicle_name: 'Creta',
    category: 'car',
    manufacturer: 'Hyundai'
  }
  {
    vehicle_name: 'Ashok Leyland Viking',
    category: 'bus',
    manufacturer: 'Ashok Leyland'
  }
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called 'animal' and *write* a MongoDB query to select database as 'animal'.

```
> use animal
< switched to db animal
```

2. Write a MongoDB query to display all the databases.

```
> show dbs
< admin      40.00 KiB
  animal     80.00 KiB
  config    108.00 KiB
  local      88.00 KiB
  test       72.00 KiB
  vehicles   80.00 KiB
```

3. Create a collection called 'wild_animals'.(use capping) and Create a collection called 'domestic_animals'.

```
> db.createCollection("wild_animals", { capped: true, size: 1048576 })
  db.createCollection("domestic_animals")
< { ok: 1 }
```

4. Add 5 wild_animal details to the collection named 'wild_animals'. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
db.wild_animals.insertMany([
  {
    animal_name: "Royal Bengal Tiger",
    nature: "harm",
    favorite_foods: ["deer", "wild boar", "buffalo"],
    care_taker_name: "Ravi Kumar",
    life_span: 15,
    timestamp: ISODate("2023-05-10"),
    expenses: 250000
  },
  {
    animal_name: "African Lion",
    nature: "harm",
    favorite_foods: ["zebra", "antelope"],
    care_taker_name: "Priya Sharma",
    life_span: 12,
    timestamp: ISODate("2022-08-15"),
    expenses: 220000
  },
  {
    animal_name: "Indian Elephant",
    nature: "harmless",
    favorite_foods: ["grass", "leaves", "fruits"],
    care_taker_name: "Anil Verma",
    life_span: 60,
    timestamp: ISODate("2021-11-20"),
    expenses: 180000
  },
```

```
  {
    animal_name: "White Rhino",
    nature: "harmless",
    favorite_foods: ["grass"],
    care_taker_name: "Sita Devi",
    life_span: 45,
    timestamp: ISODate("2023-02-01"),
    expenses: 200000
  },
  {
    animal_name: "Snow Leopard",
    nature: "harm",
    favorite_foods: ["ibex", "blue sheep"],
    care_taker_name: "Ravi Kumar",
    life_span: 13,
    timestamp: ISODate("2022-12-05"),
    expenses: 190000
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b134c24fee05ee9c0e48b5'),
    '1': ObjectId('69b134c24fee05ee9c0e48b6'),
    '2': ObjectId('69b134c24fee05ee9c0e48b7'),
    '3': ObjectId('69b134c24fee05ee9c0e48b8'),
    '4': ObjectId('69b134c24fee05ee9c0e48b9')
  }
}
```

5. Add 5 domestic-animal details to the collection named 'domestic_animals'. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
> db.domestic_animals.insertMany([
    {
      animal_name: "Labrador",
      gender: "male",
      favorite_foods: ["chicken", "rice"],
      animal_petname: "Bruno",
      life_span: 12,
      timestamp: ISODate("2023-01-15"),
      expenses: 50000
    },
    {
      animal_name: "Persian Cat",
      gender: "female",
      favorite_foods: ["fish", "milk"],
      animal_petname: "Mimi",
      life_span: 15,
      timestamp: ISODate("2022-09-10"),
      expenses: 35000
    },
    {
      animal_name: "German Shepherd",
      gender: "male",
      favorite_foods: ["meat", "bones"],
      animal_petname: "Rex",
      life_span: 10,
      timestamp: ISODate("2023-03-20"),
      expenses: 65000
    },
```

```
    {
      animal_name: "Siamese Cat",
      gender: "female",
      favorite_foods: ["tuna", "chicken"],
      animal_petname: "Luna",
      life_span: 14,
      timestamp: ISODate("2022-07-01"),
      expenses: 40000
    },
    {
      animal_name: "Golden Retriever",
      gender: "male",
      favorite_foods: ["beef", "vegetables"],
      animal_petname: "Max",
      life_span: 11,
      timestamp: ISODate("2023-04-12"),
      expenses: 55000
    }
  ])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b134cd4fee05ee9c0e48ba'),
    '1': ObjectId('69b134cd4fee05ee9c0e48bb'),
    '2': ObjectId('69b134cd4fee05ee9c0e48bc'),
    '3': ObjectId('69b134cd4fee05ee9c0e48bd'),
    '4': ObjectId('69b134cd4fee05ee9c0e48be')
  }
}
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
> db.wild_animals.find().pretty()
  db.domestic_animals.find().pretty()
< {
    _id: ObjectId('69b134cd4fee05ee9c0e48ba'),
    animal_name: 'Labrador',
    gender: 'male',
    favorite_foods: [
      'chicken',
      'rice'
    ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: 2023-01-15T00:00:00.000Z,
    expenses: 50000
  }
  {
    _id: ObjectId('69b134cd4fee05ee9c0e48bb'),
    animal_name: 'Persian Cat',
    gender: 'female',
    favorite_foods: [
      'fish',
      'milk'
    ],
    animal_petname: 'Mimi',
    life_span: 15,
    timestamp: 2022-09-10T00:00:00.000Z,
    expenses: 35000
  }
```

```
{
  _id: ObjectId('69b134cd4fee05ee9c0e48bc'),
  animal_name: 'German Shepherd',
  gender: 'male',
  favorite_foods: [
    'meat',
    'bones'
  ],
  animal_petname: 'Rex',
  life_span: 10,
  timestamp: 2023-03-20T00:00:00.000Z,
  expenses: 65000
}
{
  _id: ObjectId('69b134cd4fee05ee9c0e48bd'),
  animal_name: 'Siamese Cat',
  gender: 'female',
  favorite_foods: [
    'tuna',
    'chicken'
  ],
  animal_petname: 'Luna',
  life_span: 14,
  timestamp: 2022-07-01T00:00:00.000Z,
  expenses: 40000
}
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
> db.wild_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
  db.domestic_animals.find({}, { animal_name: 1, expenses: 1, _id: 0 })
< {
    animal_name: 'Labrador',
    expenses: 50000
  }
  {
    animal_name: 'Persian Cat',
    expenses: 35000
  }
  {
    animal_name: 'German Shepherd',
    expenses: 65000
  }
  {
    animal_name: 'Siamese Cat',
    expenses: 40000
  }
  {
    animal_name: 'Golden Retriever',
    expenses: 55000
  }
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
> db.domestic_animals.find({ life_span: 12 }).pretty()
< {
    _id: ObjectId('69b134cd4fee05ee9c0e48ba'),
    animal_name: 'Labrador',
    gender: 'male',
    favorite_foods: [
      'chicken',
      'rice'
    ],
    animal_petname: 'Bruno',
    life_span: 12,
    timestamp: 2023-01-15T00:00:00.000Z,
    expenses: 50000
  }
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
db.wild_animals.find({ care_taker_name: "Ravi Kumar" }).pretty()
{
  _id: ObjectId('69b134c24fee05ee9c0e48b5'),
  animal_name: 'Royal Bengal Tiger',
  nature: 'harm',
  favorite_foods: [
    'deer',
    'wild boar',
    'buffalo'
  ],
  care_taker_name: 'Ravi Kumar',
  life_span: 15,
  timestamp: 2023-05-10T00:00:00.000Z,
  expenses: 250000
}
{
  _id: ObjectId('69b134c24fee05ee9c0e48b9'),
  animal_name: 'Snow Leopard',
  nature: 'harm',
  favorite_foods: [
    'ibex',
    'blue sheep'
  ],
  care_taker_name: 'Ravi Kumar',
  life_span: 13,
  timestamp: 2022-12-05T00:00:00.000Z,
  expenses: 190000
}
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years

```
> db.wild_animals.find(
    { life_span: { $gt: 5 } },
    { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
  ).pretty()

  db.domestic_animals.find(
    { life_span: { $gt: 5 } },
    { animal_name: 1, favorite_foods: 1, expenses: 1, _id: 0 }
  ).pretty()
< {
    animal_name: 'Labrador',
    favorite_foods: [
      'chicken',
      'rice'
    ],
    expenses: 50000
  }
  {
    animal_name: 'Persian Cat',
    favorite_foods: [
      'fish',
      'milk'
    ],
    expenses: 35000
  }
```

```
{
  animal_name: 'German Shepherd',
  favorite_foods: [
    'meat',
    'bones'
  ],
  expenses: 65000
}
{
  animal_name: 'Siamese Cat',
  favorite_foods: [
    'tuna',
    'chicken'
  ],
  expenses: 40000
}
{
  animal_name: 'Golden Retriever',
  favorite_foods: [
    'beef',
    'vegetables'
  ],
  expenses: 55000
}
```